



Laboratory Work 1

Finding Minima using Static Optimization

Lab Goal: *To find the global minimum of the given cost function using Static Optimization*

Student ID:

➤ *Muhammad Bilal (474335)*

Lab Variant: 28

Teacher: Aleksey Paramonov

1. Introduction

In this laboratory work we have to find the minima of the given cost function under different constraints and conditions using different kind of methods using static optimization, we have a cost function and a constraint as per the variant number 28,

$$J(x, u) = 8x^2 + 3u^2 + 7xu + 2x + 6u - 5 \rightarrow \text{Cost Function}$$

$$c(x, u) = -2x + 5u - 4 \rightarrow \text{Constraint}$$

1.1 Unconstraint minimum

For finding the unconstrained minimum we will use the partial differentiation to the cost function to form 2 equations and then simultaneously solve them to get the optimal values for the minimum,

$$\text{grad}J(x, u) = \nabla J(x, u) = \begin{bmatrix} \frac{\partial J(x, u)}{\partial x} \\ \frac{\partial J(x, u)}{\partial u} \end{bmatrix}$$

So this results in 2 different equations with respect to x and u,

$$\begin{cases} 16x + 7u + 2 = 0 \\ 7x + 6u + 6 = 0 \end{cases}$$

```

syms x u

J = 8*x.^2 + 3*u.^2 + 7*x.*u + 2*x + 6*u - 5;

dJ_dx = diff(J, x);
dJ_du = diff(J, u);

sol = solve([dJ_dx == 0, dJ_du == 0], [x, u]);

x_opt = double(sol.x);
u_opt = double(sol.u);

fprintf('Unconstrained minimum at:\n');
disp( x_opt);
disp( u_opt);

```

Figure 1. Calculating unconstraint minimum

So the resulting minimum values are,

$$x = 0.6383, \quad u = -1.7447, \quad J = -9.595$$

1.2 Equality constraint minimum

In this case we have a constraint on our system but the type of the constraint is equality constraint so our constraint will be,

$$c(x, u) \rightarrow -2x + 5u - 4 = 0$$

For this purpose we will use the lagrangian function,

$$L(x, u, \lambda) = J(x, u) + \lambda c(x, u)$$

Where λ is the lagrangian multiplier

By solving our system will take the shape,

$$\begin{cases} -2x + 5u = 4 \\ 16x + 7u - 2\lambda = -2 \\ 7x + 6u + 5\lambda = -6 \end{cases}$$

```
syms x u lambda

J = 8*x^2 + 3*u^2 + 7*x*u + 2*x + 6*u - 5;

c = -2*x + 5*u - 4;

L = J + lambda * c;

dL_dx = diff(L, x);
dL_du = diff(L, u);
dL_dl = diff(L, lambda);

sol = solve([dL_dx == 0, dL_du == 0, dL_dl == 0], [x, u, lambda]);

x_sol = double(sol.x);
u_sol = double(sol.u);

fprintf('Equality-constrained minimum:\n');
disp( x_sol);
disp(u_sol);
```

Figure 2. Calculating minimum with equality constraint

The resulting values are,

$$x = -0.5284, \quad u = 0.5887, \quad J = -1.4287$$

1.3 Inequality constraint minimum

In this case the constraint has an inequality in the form,

$$c(x, u) \rightarrow -2x + 5u - 4 \leq 0$$

So the system will take the form,

$$\begin{cases} -2x + 5u - 4 \leq 0 \\ 16x + 7u - 2\lambda = -2 \\ 7x + 6u + 5\lambda = -6 \end{cases}$$

Since the unconstrained minimum satisfies the condition $-2x + 5u - 4 \leq 0$

So it will remain valid

For the inequality constraint we also have 2 more conditions,

- Kuhn tucker condition $\lambda \geq 0$
- Complementary slackness condition $\lambda c(x, u) = 0$

```

syms x u lambda

J = 8*x^2 + 3*u^2 + 7*x*u + 2*x + 6*u - 5;

c = -2*x + 5*u - 4;

L = J + lambda * c;

dL_dx = diff(L, x);
dL_du = diff(L, u);
comp_cond = lambda * c;

sol = solve([dL_dx == 0, dL_du == 0, comp_cond == 0, c <= 0, lambda >= 0], ...
            [x, u, lambda], 'Real', true);

x_vals = double(sol.x);
u_vals = double(sol.u);
lambda_vals = double(sol.lambda);

fprintf('Inequality-constrained minimum(s):\n');
for i = 1:length(x_vals)
    |
    disp(x_vals(i))
    disp(u_vals(i))
    disp(lambda_vals(i))
end

```

Figure 3. Calculating Inequality constraint

So the resulting values are,

$$x = 0.6383, \quad u = -1.7447, \quad J = -9.595$$

1.4 Newton Raphsen Method

In this part we will use the gradient method instead of the normal static optimization, this is a gradient descent method that uses the Heissan matrix for the calculation of the minimum,

$$\bar{x}^{(n+1)} = \bar{x}^n - H^{-1}(\bar{x}^n) gradJ(\bar{x}^n)$$

Where,

$$\bar{x}^n = \begin{bmatrix} x(n) \\ u(n) \end{bmatrix}$$

$$gradJ(\bar{x}^n) = \begin{bmatrix} \frac{\partial J(\bar{x}^n)}{\partial x} \\ \frac{\partial J(\bar{x}^n)}{\partial u} \end{bmatrix}$$

$$H(\bar{x}^n) = \begin{bmatrix} \frac{\partial}{\partial x} \left(\frac{\partial J(\bar{x}^n)}{\partial x} \right) & \frac{\partial}{\partial x} \left(\frac{\partial J(\bar{x}^n)}{\partial u} \right) \\ \frac{\partial}{\partial u} \left(\frac{\partial J(\bar{x}^n)}{\partial x} \right) & \frac{\partial}{\partial u} \left(\frac{\partial J(\bar{x}^n)}{\partial u} \right) \end{bmatrix}$$

Where the Heissan Matrix is a matrix of second order partial derivatives

```

syms x u

J = 8*x^2 + 3*u^2 + 7*x*u + 2*x + 6*u - 5;

gradJ = gradient(J, [x, u]);
HessJ = hessian(J, [x, u]);

gradF = matlabFunction(gradJ, 'Vars', {x, u});
hessF = matlabFunction(HessJ, 'Vars', {x, u});

xk = [0; 0];
tol = 1e-6;
max_iter = 10;

for k = 1:max_iter
    g = gradF(xk(1), xk(2));
    H = hessF(xk(1), xk(2));

    step = H \ g;
    xk = xk - step;

    grad_norm = norm(g);
    fprintf('%4d | %7.4f %7.4f %7.4f\n', k, xk(1), xk(2), grad_norm);

    if grad_norm < tol
        break;
    end
end

disp(xk(1));
disp(xk(2));

```

Figure 4. Calculating minimum using newton raphsen

```

[X,U] = meshgrid(-2:0.1:2, -3:0.1:1);
J_val = 8*X.^2 + 3*U.^2 + 7*X.*U + 2*X + 6*U -5;

figure;
surf(X, U, J_val); hold on;
plot3(path(1,:), path(2,:), ...
    arrayfun(@(x,u) 8*x^2+3*u^2+7*x*u+2*x+6*u-5, path(1,:), path(2,:)), ...
    'r-o', 'LineWidth', 2, 'MarkerFaceColor', 'r');
xlabel('x'); ylabel('u'); zlabel('J(x,u)');
title('Newton-Raphson Convergence');

```

Figure 5. Plotting the 3d plot for newton raphsen

The resulting values for the minimum are,

$$x = 0.6383, \quad u = -1.7447, \quad J = -9.595$$

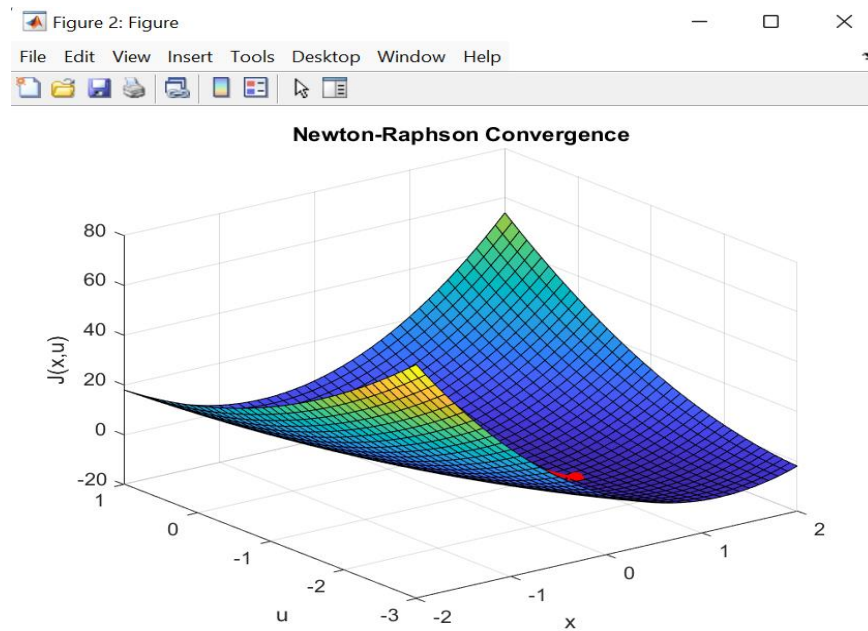


Figure 6. Plot for newton raphsen convergence

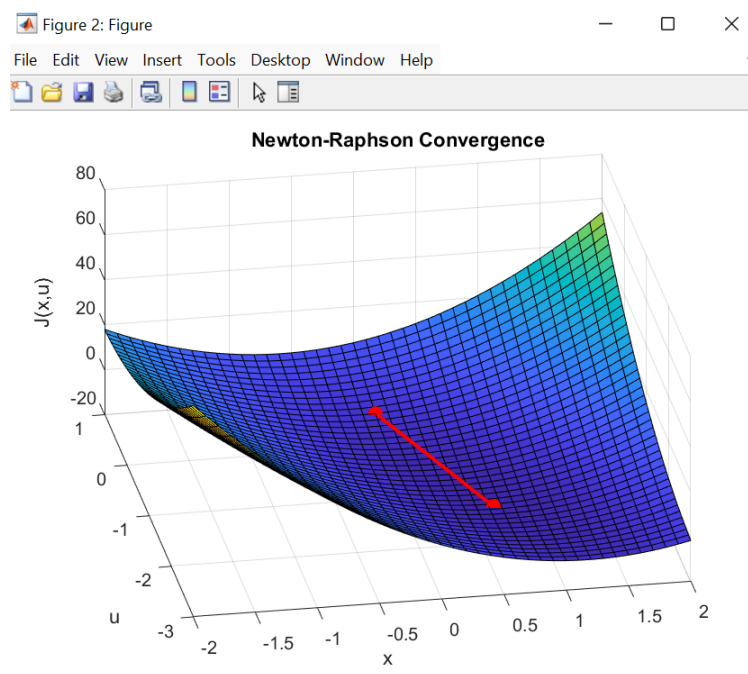


Figure 7. Newton raphsen plot (Top view)

1.5 Steepest Descent Method

Steepest descent method is another form of the gradient descent method that uses the learning rate γ to converge towards the minimum,

$$\bar{x}^{(n+1)} = \bar{x}^n - \gamma \text{grad}J(\bar{x}^n)$$

```
syms x u;  
J = 8*x^2 + 3*u^2 + 7*x*u + 2*x + 6*u - 5;  
  
gradJ = gradient(J, [x, u]);  
gradF = matlabFunction(gradJ, 'Vars', {x, u});  
  
xk = [0; 0];  
gamma = 0.01;  
path = xk;  
  
for k = 1:100  
    g = gradF(xk(1), xk(2));  
    xk = xk - gamma*g;  
    path = [path xk];  
    if norm(g) < 1e-6, break; end  
end
```

Figure 8. Calculation minimum using steepest descent

The resulting values are,

$$x = 0.6383, \quad u = -1.7447, \quad J = -9.595$$

Let's check the convergence on 2 different gamma,

```
[X,U] = meshgrid(-2:0.1:2, -3:0.1:1);
J_val = 8*X.^2 + 3*U.^2 + 7*X.*U + 2*X + 6*U -5;

figure;
surf(X, U, J_val, 'EdgeColor', 'none'); hold on;
plot3(path(1,:), path(2,:), ...
      arrayfun(@(x,u) 8*x^2+3*u^2+7*x*u+2*x+6*u-5, path(1,:), path(2,:)), ...
      'r-o', 'LineWidth', 1.5, 'MarkerFaceColor', 'r', 'MarkerSize', 4);
xlabel('x'); ylabel('u'); zlabel('J(x,u)');
title(sprintf('Steepest Descent (γ = %.2f)', gamma));
grid on;
```

Figure 9. Plotting the convergence

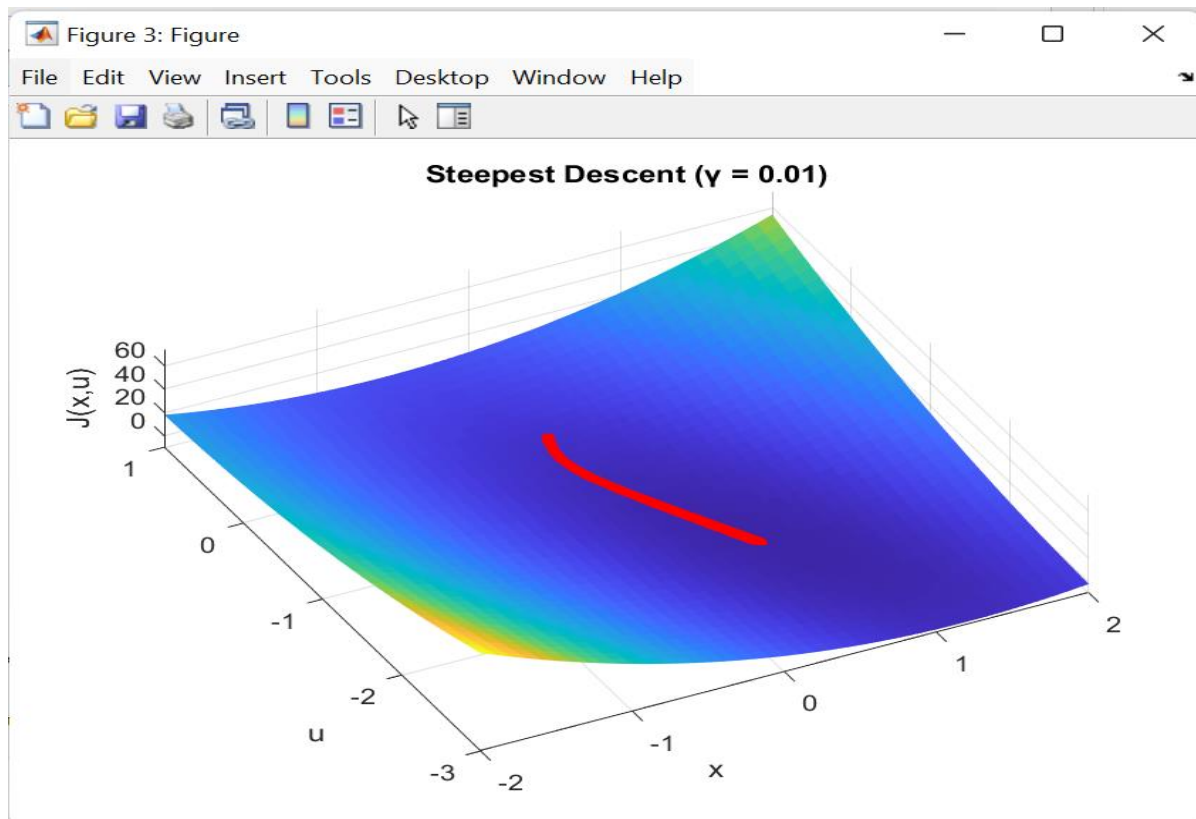


Figure 10. Steepest Descent convergence at gamma 0.01

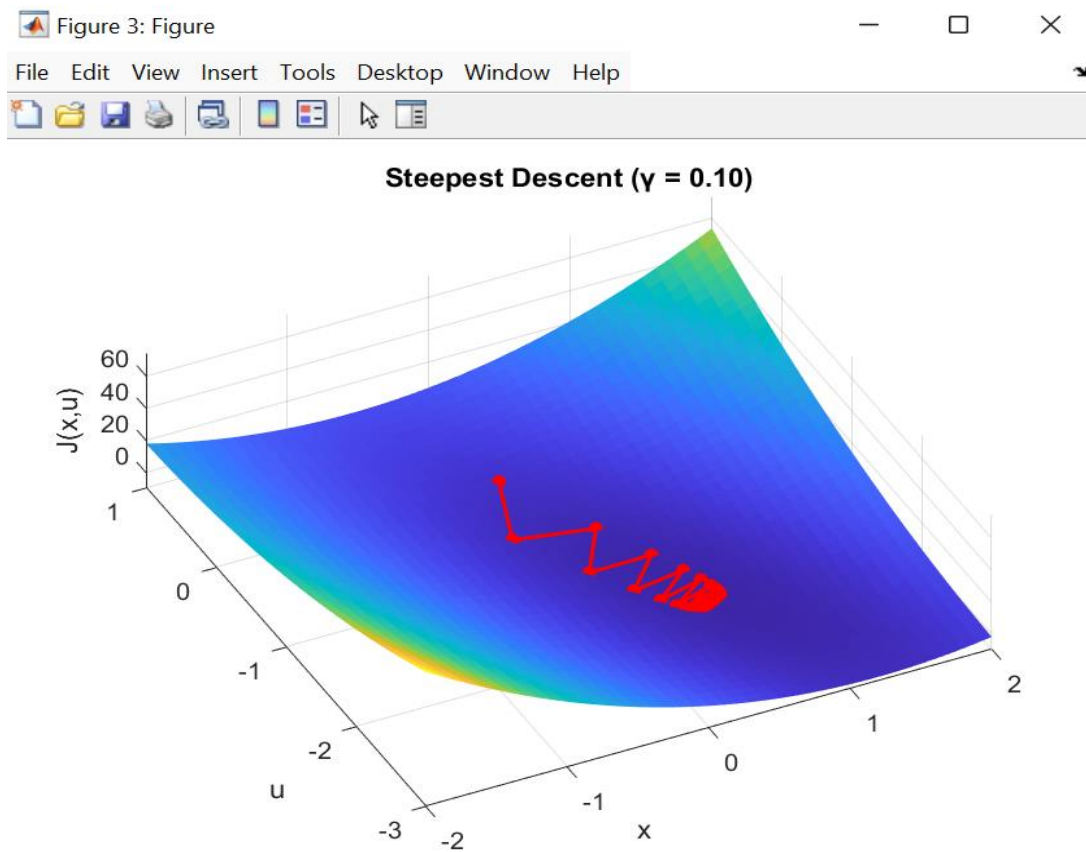


Figure 11. Steepest Descent convergence using gamma 0.1

We can clearly observe that by using a lower value of gamma the system converges smoothly towards the minimum values but taking a bit longer time, whereas by using a higher gamma the system converges fast but with higher oscillations.

Conclusion

We can conclude that the normal static optimization methods analytically solves the equation and are fast for smaller or lower dimensional systems since we need to calculate the heissan, while the gradient descent method needs only the first gradient to start converging towards the minimum values but it is a bit slower and need more iterations to converge.